

Table of Contents

1. [Introduction](#)
2. [Database Normalization](#)
 - [Introduction to Normalization](#)
 - [1NF](#)
 - [2NF](#)
 - [3NF](#)
3. [ACID Properties](#)
 - [Introduction to ACID properties](#)
 - [Atomicity](#)
 - [Consistency](#)
 - [Isolation](#)
 - [Durability](#)

INTRO

Database normalization and ACID properties are foundational concepts in database management system that ensure data integrity, efficiency and reliability.

In this document, I have discussed in details the concepts of database normalization, and the ACID properties which are essential components in enhancing efficiency in data access in databases as well as ensuring that there is redundancy and a quick recovery from failure even in cases of System failure, Power loss or crashes in the system.

DATABASE NORMALIZATION

INTRODUCTION

Database normalization refers to the process of organizing data in a the database so that it is stored efficiently and without unnecessary duplication.

Normalization helps break large tables into small related tables while defining clear relationships between them thus acheive:

- Create Update Read and Delete operations are done without errors
- Each peice of data is stored only once (no duplicates)
- Prevent un intentional data loss

FIRST NORMAL FORM(1NF)

In the first normalization, the aim is to **ensure atomicity of the data stored in each cell** thus each cell stores only 1 piece of data.

There are no repeating groups or arrays of data; the data is atomic(indivisible).

For example basing in a database that stores the order information of a FRUIT-PARLOUR business that sells fruits.

Table in Zeroth Normal form:

ORDER_INFO table

Order_id	customer_name	items_bought
1	Mwihaki	Mango
2	John	Melon
3	Kefa	Orange, Pinaapple
4	David	Avocado, Pawpaw

To remove the multiple inputs in the `items_bought` column for `order_id` 3, 4, The table is noramlized to First Normal form resulting to:

Table in First Normal Form:

ORDER_INFO table

Order_id	customer_name	items_bought
1	Mwihaki	Mango
2	John	Melon
3	Kefa	Orange
3	Kefa	Pineapple
4	David	Avocado
4	David	Pawpaw

SECOND NORMAL FORM(2NF)

The Second Normalization(2NF) build on top of the First Normal Form by removing **partial dependancies** that is, all non-key columns must depend only on the Primary-Key column.

For example: introducing an Order_date column in the 1NF-ORDER_INFO table gives:

Order_id	customer_name	items_bought	Order_date
1	Mwihaki	Mango	2025-06-01
2	John	Melon	2025-06-03
3	Kefa	Orange	2025-06-05
3	Kefa	Pineapple	2025-06-07
4	David	Avocado	2025-06-07
4	David	Pawpaw	2025-06-08

The 2NF would require the table to be split into two **Orders** table and **Order_details** table

Orders table:

Order_id	customer_name	Order_date
1	Mwihaki	2025-06-01
2	John	2025-06-03
3	Kefa	2025-06-05
3	Kefa	2025-06-07
4	David	2025-06-07
4	David	2025-06-08

Order_details table

Order_id	items_bought
1	Mango
2	Melon
3	Orange
3	Pineapple
4	Avocado
4	Pawpaw

THIRD NORMAL FORM(3NF)

The Third Normal Form (3NF) builds on top of 2NF by removing **transitive dependencies**. A transitive dependency occurs when a non-key column depends on another non-key column, rather than depending directly on the primary key.

3NF ensures that:

- Every non-key column must depend only on the primary key, and not on any other non-key column.
- This further reduces redundancy and makes the database easier to maintain.

For Example (Continuing from 2NF):

Suppose we add a `customer_address` column to the `orders` table:

Order_id	customer_name	Order_date	customer_address
1	Mwihaki	2025-06-01	Nairobi
2	John	2025-06-03	Nakuru
3	Kefa	2025-06-05	Eldoret
4	Kefa	2025-06-07	Eldoret
5	David	2025-06-07	Kisumu
6	David	2025-06-08	Kisumu

In the 2NF table above `customer_address` depends on `customer_name` , not directly on `order_id` . This is a transitive dependency.

To achieve 3NF, we split the table further:

orders table:

Order_id	customer_name	Order_date
1	Mwihaki	2025-06-01
2	John	2025-06-03
3	Kefa	2025-06-05
3	Kefa	2025-06-07
4	David	2025-06-07
4	David	2025-06-08

Order_details table:

Order_id	items_bought
1	Mango
2	Melon

Order_id	items_bought
3	Orange
3	Pineaapple
4	Avocado
4	Pawpaw

Customers table:

customer_name	customer_address
Mwihaki	Nairobi
John	Nakuru
Kefa	Eldoret
David	Kisumu

`customer_address` is stored only once for each customer, and all non-key columns depend only on the primary key of their respective tables.

ACID PROPERTIES

INTRODUCTION

ACID is an acronym for a set of properties that ensure that database transactions are processed reliably. The properties are Atomicity, Consistency, Isolation, and Durability. Implementing these properties in a database system is crucial for maintaining data integrity, especially in systems that involve concurrent transactions and critical data operations.

ATOMICITY

Atomicity ensures that each database transaction is treated as a single, indivisible unit. This means that either all operations within the transaction are completed successfully, or none of them are applied. If any part of the transaction fails, the entire transaction is rolled back, leaving the database unchanged.

For Example:

Transferring money from a bank account to a friend's account using online banking. The transaction involves two steps:

1. Deducting the amount from your account.
2. Adding the amount to your friend's account.

With atomicity, both steps must succeed for the transaction to be complete. If the system crashes after deducting money from your account but before adding it to your friend's account, atomicity ensures that the deduction is undone, so no money is lost or created. The transaction is either fully completed or not done at all.

CONSISTENCY

Consistency ensures that a database remains in a valid state before and after a transaction. Any transaction must take the database from one valid state to another, following all defined rules, constraints, and triggers.

For Example:

In a university database where every student must be assigned to an existing course. If you try to enroll a student in a course that doesn't exist, the transaction will fail, and the database will remain unchanged. Consistency guarantees that the database cannot end up with invalid or corrupt data, such as students enrolled in non-existent courses.

ISOLATION

Isolation ensures that transactions are executed independently of one another. The intermediate state of a transaction is invisible to other transactions, so concurrent transactions do not interfere with each other.

For Example:

Two people booking the last available seat on a flight at the same time. Isolation ensures that only one booking will succeed, and the other will be informed that the seat is no longer available. This prevents double-booking and ensures that each transaction is processed as if it were the only one running at that moment.

DURABILITY

Durability guarantees that once a transaction has been committed, its changes are permanent, even in the event of a system crash or power failure. The database will remember the committed changes no matter what happens.

For Example:

After you successfully transfer money to your friend's account and receive a confirmation, durability ensures that the transaction is saved. Even if the bank's server crashes immediately after, your transaction will not be lost. When the system is restored, the database will still reflect the completed transfer.